

Programación

III – TP Final

Food Store

- Alumnos:
 - Nahuel Urciuoli Zabala – Nahuel_zabala@live.com
- Título del Proyecto:
 - Food Store - Sistema Integral de Gestión y Venta de Alimentos
- Fecha de entrega:
 - 29/06/2025

Índice

Marco Teórico.....	3
Decisiones Técnicas y Arquitectura del Sistema.....	3
FRONT-END.....	3
BACK-END	9
Dificultades Encontradas y Soluciones Aplicadas	19
Bibliografía	19

Marco Teórico

Java 23 & Spring Boot: Se utilizó Java como lenguaje principal en el backend debido a su fuerte tipado y madurez en entornos corporativos. Spring Boot simplificó la configuración inicial mediante el uso de starter-packs, permitiendo levantar una API REST funcional de manera ágil.

JPA / Hibernate: Como solución de persistencia (ORM), se implementó Jakarta Persistence API (JPA) a través de Hibernate. Esto permitió abstraer las consultas SQL nativas, mapeando las tablas de la base de datos directamente a clases de Java (Entidades), optimizando la mantenibilidad del código.

H2 Database: Durante la etapa de desarrollo y testing se optó por una base de datos H2 en memoria. Esto facilitó pruebas rápidas sin necesidad de configuraciones complejas de servidores locales, asegurando un entorno limpio en cada ejecución.

Vite & Frontend Stack: En el desarrollo de la interfaz de usuario se utilizó Vite como empaquetador debido a su velocidad de compilación y recarga en caliente (HMR), consumiendo la API de Spring mediante peticiones asíncronas (Fetch API).

Decisiones Técnicas y Arquitectura del Sistema

El backend se estructuró siguiendo una arquitectura basada en repositorios:

- **Capa de Acceso a Datos (Repository):** Se implementó un `BaseRepository<T>` genérico utilizando `EntityManagerFactory`. Esto permite realizar operaciones CRUD (persist, find, merge, remove) de manera reutilizable para todas las entidades, minimizando la duplicación de código.
- **Gestión de Sesiones:** Se implementó el patrón de gestión de `EntityManager` dentro de bloques try-finally para asegurar el cierre correcto de las conexiones a la base de datos tras cada operación.
- **Gestión de Relaciones:** Debido a la naturaleza de la relación OneToMany (Lazy Loading), se implementó la estrategia de **"Inicialización Forzada"** mediante JOIN FETCH en consultas JPQL específicas cuando se requiere acceder a colecciones de hijos (productos de una categoría o pedidos de un usuario), evitando el error `LazyInitializationException`.

FRONT-END

Para el front end se usó un carrito que persiste en `LocalStorage` comparando siempre el elemento en el local Storage con los datos del JSON en todo momento.

El usuario persiste en el Session Storage

Storage	Key	Value
Local storage	usuarioLogueado	{"id":2,"nombre":"Juan","apellido":"Pérez"}
Session storage		
http://localhost:5173		

Mientras que el carrito persiste en `LocalStorage`

Storage	Key	Value
Local storage	cart	[{"producto":{"id":1,"nombre":"Hamburguesa Clásica","precio":4500,"categoria":"Comida"},"cantidad":2,"total":9000}]
Session storage		
http://localhost:5173		

Antes de iniciar cada página se verifica que la persona tenga una sesión iniciada

Si rol requerido no es el adecuado se manda a home en caso de ser Usuario se manda al home de usuario

```
export const verificarUsuario : (rolRequerido: Rol) => void = (rolRequerido: Rol) : void =>
  const user : string | null = getUser()
  if (!user) {
    console.log( message: "no existe en local");
    navigate( route: "/src/pages/auth/login/login.html");
    return;
  } else {
    const parseUser: IUser = JSON.parse(user);
    if(parseUser.rol=="ADMIN") {
      return;
    }
    if (parseUser.rol !== rolRequerido) {
      if (parseUser.rol === "USUARIO" ) {
        navigate( route: "/src/pages/store/home/home.html")
        return
      }
      else {
        navigate( route: "/src/pages/admin/home/home.html");
        return
      }
    }
  }
}
```

Ejemplo de Panel Admin

```
> import ...

verificarUsuario( rolRequerido: "ADMIN")

const productoTodos: Product[] = productos
const categoriasTodas: ICategory[] = categorias
const pedidosTodos: IPedido[] = pedidos

const barraLateral = document.querySelector( selectors: ".sidebar-menu") as HTMLDivElement;
const contenedorGeneral = document.querySelector( selectors: ".main-content") as HTMLElement;
```

SuperFood

Iniciar Sesión

Email

password

Ingresar

Cuentas de prueba:

Admin: admin@admin.com / 123456

Cliente: cliente@food.com / cliente123

Pagina home

FoodStore

Catalogo

Mis Pedidos

Carrito 1

Juan Pérez

Cerrar Sesión

CATEGORÍAS

Todos los productos

Hamburguesas

Pizzas

Empanadas

Bebidas

Papas Fritas

Buscar productos...

Ordenar por: Destacados

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

Hamburguesas

Hamburguesa Clásica

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$4500

+ Agregar

Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

Hamburguesas

Hamburguesa Doble

Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

\$6200

+ Agregar

Carne 150g, huevo, jamon, queso, lechuga y tomate

Hamburguesas

Hamburguesa Completa

Carne 150g, huevo, jamón, queso, lechuga y tomate

\$5500

+ Agregar

Mozzarella, salsa de tomate y orégano

Pizzas

Pizza Mozzarella

Mozzarella, salsa de tomate y orégano

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

Pizzas

Pizza Napolitana

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

Filtrado de producto por categoría

FoodStore

Catalogo

Mis Pedidos

Carrito 1

Juan Pérez

Cerrar Sesión

CATEGORÍAS

Todos los productos

Hamburguesas

Pizzas

Empanadas

Bebidas

Papas Fritas

Buscar productos...

Ordenar por: Destacados

Mozzarella, salsa de tomate y orégano

Pizzas

Pizza Mozzarella

Mozzarella, salsa de tomate y orégano

\$3800

+ Agregar

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

Pizzas

Pizza Napolitana

Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca

\$4500

+ Agregar

Mozzarella, cebolla caramelizada, jamón y aceitunas

Pizzas

Pizza Fugazzeta

Mozzarella, cebolla caramelizada, jamón y aceitunas

\$4800

+ Agregar

Mozzarella, pepperoni, morrón, cebolla, huevo y aceitunas

Pizzas

Pizza Especial

Mozzarella, pepperoni, morrón, cebolla, huevo y aceitunas

Filtrado por búsqueda (Descripción y nombre producto)

 hamb

Carne 150g, lechuga, tomate,
cebolla morada y mayonesa

Hamburguesas
Hamburguesa Clásica

Carne 150g, lechuga, tomate,
cebolla morada y mayonesa

\$4500

+ Agregar

Doble carne 150g c/u,
cheddar, panceta y salsa
barbacoa

Hamburguesas

Hamburguesa Doble

Doble carne 150g c/u, cheddar,
panceta y salsa barbacoa

\$6200

+ Agregar

Ordenamiento A-Z

 hamb

Ordenar por: Nombre (A-Z)



Bebidas
Agua Mineral 500ml
Agua mineral sin gas 500ml

\$1000

+ Agregar



Papas Fritas
Aros de Cebolla
Aros de cebolla empanizados,
servidos con salsa barbacoa

\$2800

+ Agregar



Bebidas
Cerveza Artesanal 473ml
Cerveza artesanal rubia 473ml

\$2500

+ Agregar

Precio

hamb

Ordenar por: Precio: Menor a Mayor



Bebidas

Agua Mineral 500ml

Agua mineral sin gas 500ml

\$1000

+ Agregar



Empanadas

Empanada Jamón y Queso

Jamón cocido y mozzarella

\$1100

+ Agregar



Empanadas

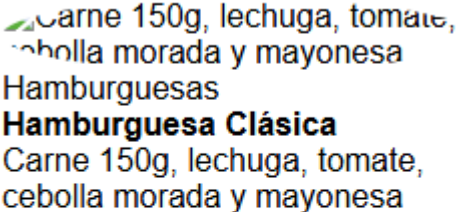
Empanada de Verdura

Espinaca, acelga, ricota y nuez moscada

\$1100

+ Agregar

Producto sin stock no permite agregar al carrito



Hamburguesa Clásica

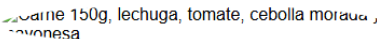
Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$4500

Sin stock

Carrito con total y agregar más productos

Mi Carrito

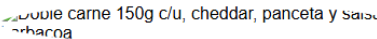


Hamburguesa Clásica

Precio: \$4500

- 3 +

Eliminar

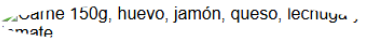


Hamburguesa Doble

Precio: \$6200

- 1 +

Eliminar



Hamburguesa Completa

Precio: \$5500

- 1 +

Eliminar

Resumen

Subtotal:

\$25200

Total:

\$25200

Finalizar Compra

Boton de checkout desactivado en esta version

Vaciar carrito

Menús pedidos con filtro según estado

FoodStore [Catalogo](#) [Mis Pedidos](#) [Carrito 0](#) [Juan Pérez](#) [Cerrar Sesión](#)

Mis Pedidos

Todos ▾

Pedido #1 ENTREGADO

📅 2026-06-08

• Hamburguesa Clásica (x2) • Agua Mineral 500ml (x2)

🛒 2 producto(s) **\$9600.00**

Pedido #2 EN PREPARACION

📅 2026-06-09

• Hamburguesa Doble (x1) • Papas Fritas Clásicas (x2)
• Pizza Especial (x1)

🛒 3 producto(s) **\$16400.00**

Pedido #3 PENDIENTE

📅 2026-06-10

• Empanada de Carne (x3) • Cerveza Artesanal 473ml (x1)

🛒 2 producto(s) **\$6100.00**

Al hacer clic se abre modal para desplegar detalles

Mis Pedidos Todos

Pedido #1 ENTREGADO

📅 2026-06-08

• Hamburguesa Clásica (x2) • Agua Mineral 500ml (x2)

🛒 2 producto(s) **\$9600.00**

Pedido #2 EN PREPARACION

📅 2026-06-09

• Hamburguesa Doble (x1) • Papas Fritas Clásicas (x2)
• Pizza Especial (x1)

🛒 3 producto(s) **\$16400.00**

Pedido #3 PENDIENTE

📅 2026-06-10

• Empanada de Carne (x3) • Cerveza Artesanal 473ml (x1)

🛒 2 producto(s) **\$6100.00**

ENTREGADO

📅 2026-06-08

📍 **Información de Entrega**

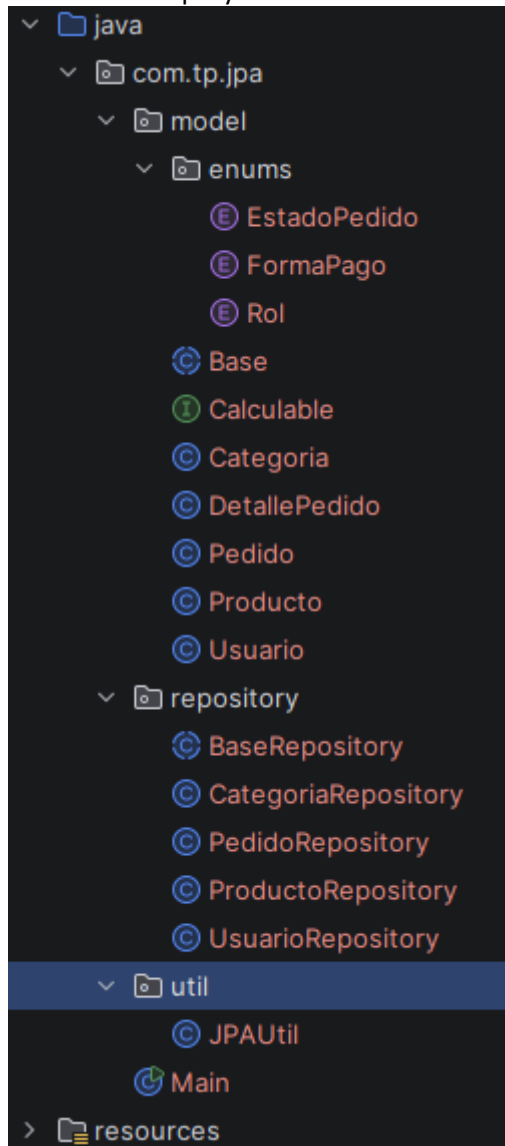
Teléfono: 1198765432
Método de pago: 💳 TARJETA

🛒 **Productos**

Hamburguesa Clásica	\$9000.00
Cantidad: 2 × \$4500.00	
Agua Mineral 500ml	\$600.00
Cantidad: 2 × \$1000.00	
Total:	\$9600.00

🏆 **Tu pedido está siendo procesado**
Te notificaremos cuando sea confirmado.

Estructura del proyecto



Repositorio Base (del cual es extendido por las otras entidades)

```
public abstract class Base {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(name = "eliminado", nullable = false)  
    @Builder.Default  
    private boolean eliminado = false;  
  
    @Column(name = "created_at", updatable = false)  
    private LocalDateTime createdAt;  
  
    @PrePersist  
    protected void prePersist() {  
        if (this.createdAt == null) {  
            this.createdAt = LocalDateTime.now();  
        }  
    }  
}
```

Categoria:

```
public class Categoria extends Base {  
  
    @EqualsAndHashCode.Include  
    @Column(name = "nombre", nullable = false, unique = true, length = 100)  
    private String nombre;  
  
    @Column(name = "descripcion", length = 500)  
    private String descripcion;  
  
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)  
    @JoinColumn(name = "categoria_id")  
    @Builder.Default  
    private Set<Producto> productos = new HashSet<>();  
}
```

Detalle Pedido:

```
public class DetallePedido extends Base {

    @Column(name = "cantidad", nullable = false)
    private Integer cantidad;

    @Column(name = "subtotal", nullable = false)
    private Double subtotal;

    @EqualsAndHashCode.Include
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "producto_id", nullable = false)
    private Producto producto;

    public void calcularSubtotal() { 2 usages
        if (this.producto != null && this.cantidad != null) {
            this.subtotal = this.producto.getPrecio() * this.cantidad;
        }
    }
}
```

Pedido:

```
public class Pedido extends Base implements Calculable {

    @Column(name = "fecha", updatable = false)
    @Builder.Default
    private LocalDate fecha = LocalDate.now();

    @Enumerated(EnumType.STRING)
    @Column(name = "estado", nullable = false, length = 30)
    @Builder.Default
    private EstadoPedido estado = EstadoPedido.PENDIENTE;

    @Column(name = "total", nullable = false)
    @Builder.Default
    private Double total = 0.0;

    @Enumerated(EnumType.STRING)
    @Column(name = "forma_pago", nullable = false, length = 20)
    private FormaPago formaPago;

    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(name = "pedido_id")
    @Builder.Default
    private Set<DetallePedido> detalles = new HashSet<>();
}
```

Funcion de añadir detalles Pedido a pedidos, si encuentra ya un producto aumenta su cantidad y actualiza subtotal

```
public void addDetallePedido(int cantidad, Producto producto) { 1 usage
    Optional<DetallePedido> detalleExistente = this.detalles.stream()
        .filter( DetallePedido d -> d.getProducto().getId().equals(producto.getId()))
        .findFirst();

    if (detalleExistente.isPresent()) {
        DetallePedido detalle = detalleExistente.get();
        int nuevaCantidad = detalle.getCantidad() + cantidad;
        detalle.setCantidad(nuevaCantidad);
        detalle.calcularSubtotal();
        this.total+=detalle.getSubtotal();
    } else {
        DetallePedido nuevoDetalle = DetallePedido.builder() DetallePedidoBuilder<capture of ?,
            .cantidad(cantidad) capture of ?
            .producto(producto)
            .build();
        nuevoDetalle.calcularSubtotal();
        this.detalles.add(nuevoDetalle);
        this.total+=nuevoDetalle.getSubtotal();
    }
}
```

Función de pedido para actualizar Total de pedido

```
@Override 1 usage
public void calcularTotal() {
    double acumulador = 0.0;
    for (DetallePedido detalle : detalles) {
        if (detalle.getSubtotal() != null) {
            acumulador += detalle.getSubtotal();
        }
    }
    this.total = acumulador;
}
```

Producto

```
public class Producto extends Base {  
  
    @EqualsAndHashCode.Include  
    @Column(name = "nombre", nullable = false, length = 100)  
    private String nombre;  
  
    @Column(name = "precio", nullable = false)  
    private Double precio;  
  
    @Column(name = "descripcion", length = 500)  
    private String descripcion;  
  
    @Column(name = "stock", nullable = false)  
    private Integer stock;  
  
    @Column(name = "imagen")  
    private String imagen;  
  
    @Builder.Default  
    @Column(name = "disponible")  
    private Boolean disponible = Boolean.TRUE;  
}
```

Usuario:

```
public class Usuario extends Base {

    @Column(name = "nombre", nullable = false, length = 50)
    private String nombre;

    @Column(name = "apellido", nullable = false, length = 50)
    private String apellido;

    @EqualsAndHashCode.Include
    @Column(name = "mail", nullable = false, unique = true, length = 150)
    private String mail;

    @Column(name = "celular", length = 20)
    private String celular;

    @Column(name = "contrasena")
    private String contraseña;

    @Enumerated(EnumType.STRING)
    @Column(name = "rol", nullable = false, length = 20)
    @Builder.Default
    private Rol rol = Rol.USUARIO;

    @Builder.Default
    @OneToMany(cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    @JoinColumn(
        name="usuario_id"
    )
    private List<Pedido> pedidos = new ArrayList<>();

    public void addPedido(Pedido pedido) { pedidos.add(pedido); }

    public void removePedido(Pedido pedido) { pedidos.remove(pedido); }
}
```

Repositorio Categoría que extiende de Base

Ya que tiene una asociación por Lazy se crea esta función para que al traer los datos de la BD los trae con todos sus productos y pueda ser trabajado posteriormente

```
public class CategoriaRepository extends BaseRepository<Categoria> { 3 usages

    public CategoriaRepository() { super(Categoria.class); }

    public Optional<Categoria> buscarConProductos(Long id) { 1 usage
        EntityManager em = emf.createEntityManager();
        try {
            Categoria categoria = em.find(Categoria.class, id);
            if (categoria != null) {
                categoria.getProductos().size();
            }

            return Optional.ofNullable(categoria);
        } finally {
            em.close();
        }
    }
}
```

Pedido Repository

Función para buscar por usuario (que usuario tiene que pedido)

Función para buscar pedido según estado

```
/**
 * Retorna los pedidos activos del usuario indicado.
 */
public List<Pedido> buscarPorUsuario(Long idUsuario) { 1 usage
    EntityManager em = emf.createEntityManager();
    try {
        String jpql = "SELECT p FROM Usuario u JOIN u.pedidos p WHERE u.id = :uid AND p.eliminado = false";
        return em.createQuery(jpql, Pedido.class)
            .setParameter(name: "uid", idUsuario)
            .getResultList();
    } finally {
        em.close();
    }
}

/**
 * Retorna los pedidos activos que coinciden con el estado indicado.
 */
public List<Pedido> buscarPorEstado(EstadoPedido estadoPedido) { 2 usages
    EntityManager em = emf.createEntityManager();
    try {
        String jpql = "SELECT p FROM Pedido p WHERE p.estado = :estado AND p.eliminado = false";
        return em.createQuery(jpql, Pedido.class)
            .setParameter(name: "estado", estadoPedido)
            .getResultList();
    } finally {
        em.close();
    }
}
```


Producto Repository

Con funciones para buscar productos según categoría

Y función para poder mostrar los Productos y sus respectivas categorías por consola

```
public class ProductoRepository extends BaseRepository<Producto> { 3 usages

    public ProductoRepository() { super(Producto.class); }

    /**
     * Retorna los productos activos que pertenecen a la categoría indicada.
     */
    //Abro el entity y escribo la consulta selecciono los productos de la categoría y muestro los los p
    public List<Producto> buscarPorCategoría(Long categoríaId) { 1 usage
        EntityManager em = emf.createEntityManager();
        try {
            String jpql = "SELECT p FROM Categoria c JOIN c.productos p WHERE c.id =:catId AND p.elimI";
            return em.createQuery(jpql, Producto.class)
                .setParameter(name: "catId", categoríaId)
                .getResultList();
        } finally {
            em.close();
        }
    }

    //Ayuda con IA) Para poder mostrar Categoría en los productos
    public List<Object[]> listarProductosConCategoría() { 1 usage
        EntityManager em = emf.createEntityManager();
        try {
            String jpql = "SELECT p, c FROM Categoria c JOIN c.productos p WHERE p.eliminado = false AT";
            return em.createQuery(jpql, Object[].class).getResultList();
        } finally {
            em.close();
        }
    }
}
```

Usuario repository

Con funciones para buscar usuarios según mail Y buscar los pedidos de los usuarios

```
public class UsuarioRepository extends BaseRepository<Usuario> { 3 usages

    public UsuarioRepository() { super(Usuario.class); }

    /**
     * Retorna el usuario activo con el mail indicado.
     */
    public Optional<Usuario> buscarPorMail(String mail) { 2 usages
        EntityManager em = emf.createEntityManager();
        try {
            //Selecciono los usuario de la lista Usuario donde el mail sea igual a los pasadors p
            String jpql = "SELECT u FROM Usuario u WHERE u.mail = :mail AND u.eliminado = false";

            List<Usuario> q = em.createQuery(jpql, Usuario.class)
                .setParameter(name: "mail", mail)
                .getResultList();

            // Si no esta vacia retorno el primer valor encontrado
            if (!q.isEmpty()) {
                return Optional.of(q.get(0));
            } else {
                return Optional.empty();
            }
        } finally {
            em.close();
        }
    }

    /**(Ayuda IA)
    public Optional<Usuario> buscarConPedidos(Long id) { 1 usage
        EntityManager em = emf.createEntityManager();
        try {
            String jpql = "SELECT u FROM Usuario u LEFT JOIN FETCH u.pedidos WHERE u.id = :id";
            List<Usuario> resultados = em.createQuery(jpql, Usuario.class)
                .setParameter(name: "id", id)
                .getResultList();
            return resultados.isEmpty() ? Optional.empty() : Optional.of(resultados.get(0));
        } catch (Exception e) {
            return Optional.empty();
        } finally {
            em.close();
        }
    }
}
```

Dificultades Encontradas y Soluciones Aplicadas

A. Gestión de LazyInitializationException

Dificultad: Al intentar acceder a colecciones (como pedidos de un usuario) fuera del EntityManager del repositorio, la aplicación lanzaba errores de sesión cerrada.

Solución: Se implementó una capa de consulta específica en los repositorios utilizando JOIN FETCH. Esto obliga a Hibernate a traer los datos asociados mientras la sesión está abierta, permitiendo su uso posterior en la capa de interfaz.

B. Estabilidad en la Consola

Dificultad: Errores como NoSuchElementException o NumberFormatException causaban el cierre inesperado del programa.

Solución: Se envolvió la lógica del menú principal en bloques try-catch robustos. Además, se utilizó el método Optional.ofNullable

C. Edición Parcial de Entidades

Dificultad: Al actualizar productos o usuarios, las entradas vacías en la consola (Scanner) causaban errores de parseo y pérdida de datos originales.

Solución: Se utilizó una estrategia de "Edición con Valores por Defecto". Se capturan las entradas como String y se verifica el .isEmpty() antes de realizar cualquier conversión. Si el usuario presiona "Enter" sin escribir, el sistema conserva el valor original que ya posee la entidad, permitiendo una experiencia de usuario fluida sin corromper los datos existentes.

Bibliografía

Hibernate. *ORM User Guide*. <https://hibernate.org/orm/documentation/>

Google. (2026). *Gemini*. <https://gemini.google.com/>

Conversaciones: <https://gemini.google.com/share/86f7cbcc7d71>